“Inter-Process Chat Application Using IPC”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

VENNELA S	**2520080018**
HARSHITHA B	**2520080007**
KRISHNASREE M	**2520080009**

Under the Esteemed Guidance of

RAGHUPATHI
Assistant Professor
Department of Artificial intelligence & Data Structures



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Inter-Process Chat Application Using IPC"** is a record of Bonafide work of **VENNELA S - 2520080018 , HARSHITHA B - 2520080007, KRISHNASREE M - 2520080009** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VENNELA S	**2520080018**
HARSHITHA B	**2520080007**
KRISHNASREE M	**2520080009**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Inter-Process Chat Application Using IPC"** is a bonafide work done and submitted by **KRISHNASREE M– 2520080009 ,**
in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University),
during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Raghupathi, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. I. Sreenath (AI&DS)**, Head of the ***Department of artificial intelligence and data structures*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

VENNELA S	**2520080018**
HARSHITHA B	**2520080007**
KRISHNASREE M	**2520080009**

Abstract

The Inter-Process Chat Application Using IPC is a simple Linux-based program that demonstrates how two independent processes can exchange messages. The application is written in C and uses named pipes, also called FIFOs, as the communication mechanism. The same program is launched in two terminal windows, one as User A and the other as User B. Two FIFO channels are used: one carries messages from A to B, and the other carries messages from B to A.

The project uses Linux/POSIX functions such as mkfifo(), open(), read(), write(), and close() to create and access the communication channels. It provides a practical demonstration of process isolation, inter-process communication, message transfer, and program termination. The application is intended as a local educational prototype; it is not an internet-based messaging service. Features and test outcomes described in this report should be verified against the final source code and actual terminal runs.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1 –5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	

		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	**System Architecture and Design**		**23–29**
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**

		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

An operating system manages program execution and provides mechanisms that allow processes to exchange information. A process is an instance of a program in execution. Processes generally have separate address spaces, which helps isolate their data and execution state. Because of this separation, processes need an operating-system-supported method when they must communicate.

Inter-Process Communication (IPC) refers to mechanisms that allow processes to exchange data or coordinate their activities. Common IPC mechanisms include pipes, named pipes, message queues, shared memory, signals, and sockets. Named pipes, or FIFOs, are useful when separate processes need a simple communication channel identified by a filesystem path.

This project, Inter-Process Chat Application Using IPC, applies the named-pipe mechanism to a small two-way chat application. The program is started as User A and User B in separate Ubuntu terminals. One FIFO carries data from A to B, while a second FIFO carries data from B to A. This practical implementation connects classroom concepts with Linux system programming in C.

The project is designed for learning and demonstration. It helps students observe how processes communicate without directly accessing each other's private memory and how system calls are used to access operating-system resources**.**

1.2 Problem Statement

Two independent processes normally operate in separate memory spaces. If they need to exchange messages, they require a suitable communication channel. A basic chat demonstration must provide a path for messages in both directions and should consider message order, unavailable peers, and clean resource closure.

The objective of this project is to develop a local chat application in which two instances of a C program exchange messages using two named pipes. The project demonstrates the use of Linux/POSIX APIs for creating, opening, reading from, writing to, and closing FIFO channels. It is a local demonstration and does not provide network-based communication.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process: A process is a program in execution. In this project, two instances of the chat program run independently as User A and User B.

Inter-Process Communication (IPC): IPC is the set of mechanisms provided by an operating system to exchange data or coordinate processes. The project uses named pipes.

Named pipe (FIFO): A FIFO is a special filesystem object used as a communication channel. Unlike an unnamed pipe, it has a name that independent processes can open.

System calls: Functions such as open(), read(), write(), close(), and mkfifo() request services from the operating system.

Concurrency: Receiving messages while a user is typing requires a concurrency strategy, such as a receiver process/thread or suitable event-driven I/O. Describe only the strategy actually present in the submitted source code.

Resource management: File descriptors should be closed when they are no longer needed. The program should also handle interruption and termination in a controlled way.

2.2 Existing Methods / Algorithms

Processes can communicate using several IPC mechanisms. Ordinary pipes are commonly used between related processes and are generally unidirectional. Named pipes (FIFOs) provide a named channel that separate processes can open. Message queues transfer structured messages through a kernel-managed queue. Shared memory allows processes to access a common memory region but usually requires additional synchronization. Sockets support local or network communication.

For this project, named pipes were selected because they are relatively simple to create and demonstrate on Linux. Two one-way FIFOs provide the two directions needed for a basic chat.

2.3 Proposed Approach

Common IPC mechanisms include pipes, named pipes, message queues, shared memory, signals, and sockets. Named pipes are useful when separate processes need a simple communication channel identified by a filesystem path.
This project is designed for learning and demonstration. It helps students observe how processes communicate without directly accessing each other's private memory and how system calls are used to access operating-system resources.

2.2 Existing Methods / Algorithms

Processes can communicate using several IPC mechanisms. Ordinary pipes are commonly used between related processes and are generally unidirectional. Named pipes (FIFOs) provide a named channel that separate processes can open. Message queues transfer structured messages through a kernel-managed queue. Shared memory allows processes to access a common memory region but usually requires additional synchronization. Sockets support local or network communication.
For this project, named pipes were selected because they are relatively simple to create and demonstrate on Linux. Two one-way FIFOs provide the two directions needed for a basic chat.

2.3 Proposed Approach

1. Create two named pipes, one for A-to-B messages and one for B-to-A messages.
2. Run the same executable in two terminal windows, identifying each instance as A or B.
3. Open the appropriate FIFO channels for each participant.
4. Read user input and write it to the outgoing FIFO.
5. Read incoming data from the other FIFO and display it in the terminal.
6. On exit, close file descriptors and clean up process resources as implemented in the source code.

2.4 Advantages of the Proposed Approach

- Simple to understand and demonstrate in an Operating Systems laboratory.
- Uses standard Linux/POSIX concepts and system calls.
- Supports two-way local communication by using two FIFOs.
- Does not require an internet connection or a remote server.
- Can be extended for a graphical interface, message logging, or additional participants.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

A basic laptop or desktop computer capable of running Ubuntu or a Linux environment is sufficient. A keyboard and display are required for terminal input and output. No specialized hardware is required.

3.2 Software Requirements

- Ubuntu/Linux environment (or WSL2 with a suitable Linux distribution).
- GCC compiler and standard build tools.

• A terminal emulator with two terminal windows or tabs.
• Text editor such as VS Code or Nano.
• C source file (chat.c) and optional Makefile.

3.3 Operating System Environment – Ubuntu / WSL

The application is intended to run in a Linux environment. Ubuntu provides POSIX APIs and filesystem support for named pipes. If Windows Subsystem for Linux (WSL) is used, commands are executed inside the Linux shell rather than the Windows Command Prompt.

3.4 Development Tools and Compilers

GCC compiles the C source file. The terminal is used to compile and launch both participants. VS Code or Nano can be used to edit the source code. A Makefile may be used to simplify repeated compilation.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is used to implement command-line input, message handling, FIFO operations, and program control. It is suitable for this project because it exposes operating-system interfaces through standard and POSIX functions.

4.2 GCC Compiler

GCC converts chat.c into an executable. Example compilation command: gcc -Wall -Wextra -std=c11 chat.c -o chat. Compiler warnings help identify potential issues in the source code.

4.3 Linux System Calls / APIs

• mkfifo(): creates a named FIFO in the filesystem.
• open(): opens a FIFO and returns a file descriptor.
• read(): reads available bytes from a file descriptor.
• write(): writes bytes to a file descriptor.
• close(): releases a file descriptor.
• fork(), waitpid(), kill(), or signal handlers: include these in the report only if they are present in the final source code.

4.4 POSIX Threads / Synchronization

Pthreads can be used to run a receiver thread concurrently with the input loop. However, the exact implementation must be described according to the submitted source code. If the program uses a child process or another I/O strategy instead of Pthreads, replace this description with the mechanism actually implemented.

4.5 Linux Commands and Utilities

• pwd and ls: inspect the current directory and its files.
• nano chat.c: edit the source file using Nano.
• gcc chat.c -o chat: compile the source file.
• ./chat A and ./chat B: launch the two chat participants.

• make and make clean: build or remove the executable if a Makefile is provided.

4.6 Other Tools / Libraries Used

The project uses the C standard library for input, output, string handling, and error reporting, together with POSIX/Linux headers for FIFO and process-related APIs as required by the implementation.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The architecture contains User A, FIFO A-to-B, FIFO B-to-A, and User B. Both users run separate instances of the same C program. The Linux kernel and filesystem provide the named-pipe mechanism.

Text representation: User A → FIFO A-to-B → User B; User B → FIFO B-to-A → User A.

5.2 System Workflow

1. The program starts and reads the participant identifier (A or B).
2. The program creates or accesses the two named FIFO paths.
3. Each participant opens the FIFO used for outgoing messages and the FIFO used for incoming messages.
4. The user types a message in the terminal.
5. The program writes the message bytes to the outgoing FIFO.
6. The other participant reads the bytes and displays the message.
7. The process closes its resources when the user exits or an error requires shutdown**.**

5.3 Flowchart / Data Flow

Start → Read participant ID → Prepare FIFO channels → Open communication channels → Start message input/receiving → Exchange messages → Exit requested or error → Close resources → End.

5.4 Module Description

Participant setup module: identifies the process as User A or User B and selects the incoming and outgoing FIFO paths.

FIFO setup module: creates the named pipes if needed and opens them for communication.

Message sending module: reads user input and transfers the message through the outgoing FIFO.

Message receiving module: reads data from the incoming FIFO and displays the received message.

Shutdown and error-handling module: handles the exit command, reports errors, and closes available resources.

5.5 Input and Output Design

Input consists of text entered by a user in the terminal and a participant identifier supplied when launching the program. Output consists of connection/status messages and incoming chat

messages displayed in the terminal. The exact exit command must match the final implementation (for example, /quit or /exit).

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The program is implemented in a C source file named chat.c. The participant argument determines whether the instance behaves as A or B. Each instance uses different incoming and outgoing FIFO paths so that messages travel in the intended direction.

6.2 Algorithm / Pseudocode

1. Begin.
2. Read and validate the participant argument A or B.
3. Set incoming and outgoing FIFO paths according to the participant.
4. Create the named pipes if required.
5. Open the FIFO channels and prepare message handling.
6. While the user has not requested exit: accept input, write outgoing messages, and display incoming messages using the implementation's receiving mechanism.
7. Close open file descriptors and terminate cleanly.
8. End.

6.3 Core C Implementation

The complete implementation should be maintained in chat.c in the project repository. The central operations are FIFO creation, opening the channels, writing outgoing message data, reading incoming data, and handling termination. Insert the final, tested source code in this section if the course documentation requires the full listing.

6.4 Linux System Calls / APIs Used

mkfifo() creates named pipes. open() obtains file descriptors for communication. read() and write() transfer bytes. close() releases descriptors. Additional process or signal APIs should be documented only if they appear in the final source**.**

6.5 Process / Thread Implementation

Two instances of the executable represent the two participants and run independently. If the implementation uses fork(), a child process, or Pthreads to separate receiving from sending, describe that mechanism exactly as present in the final code. Do not claim a thread or child process is used unless verified.

6.6 IPC / Synchronization Implementation

Two FIFOs are used because a single FIFO is treated as a one-way channel in this design. The application must coordinate channel opening and reading/writing so that both participants can exchange data. FIFO opening behaviour depends on the flags and the order in which the two processes start; test startup and peer-unavailable cases using the final code.

6.7 Memory and Resource Management

The program uses buffers to hold message data. Input should be bounded or handled safely, and reads should be checked for errors or end-of-file. File descriptors should be closed during shutdown. FIFO cleanup should be implemented carefully so that an active peer is not disrupted unexpectedly.

6.8 User Interface

The current interface is terminal-based. The user enters messages using the keyboard and reads incoming messages in the same terminal. A GUI is a possible future enhancement rather than a feature of the basic prototype.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open the Ubuntu terminal and move to the project directory. Compile with the following commands:

```
cd ~/ipc-chat
gcc -Wall -Wextra -std=c11 chat.c -o chat
```

7.2 Execution Commands

Open two terminal windows. In the first terminal, run:

```
cd ~/ipc-chat
./chat A
```

In the second terminal, run:

```
cd ~/ipc-chat
./chat B
```

Type a message in one terminal and check that it appears in the other. Reply from the second terminal to verify communication in the reverse direction. Use the exit command supported by the actual source code.

7.3 Sample Inputs and Outputs

Example input from User A: Hello B. Expected observation: User B's terminal displays the message from User A. Example reply from User B: Hello A. Expected observation: User A's terminal displays the reply. These are illustrative examples; replace them with the actual output produced by the tested program.

7.4 Screenshots of Execution

Insert genuine screenshots from the Ubuntu terminal showing: (1) successful compilation, (2) User A and User B running in separate terminals, (3) a message sent from A to B, and (4) a reply from B to A. Screenshots should be captured from the actual implementation and should not be represented as real test evidence if they are only illustrative.

7.5 Demonstration of OS Concepts

The demonstration shows process execution, IPC through named pipes, system calls for file operations, message transfer between separate processes, and resource closure. It also highlights practical considerations such as FIFO opening behaviour and orderly termination.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The Inter-Process Chat Application Using IPC is a small Linux C project that demonstrates how two independent processes exchange text messages using named pipes. Two FIFO channels provide communication in opposite directions.

8.2 Objectives Achieved

• Designed a simple local chat model using two process instances.
• Applied named pipes as an IPC mechanism.
• Used C and Linux/POSIX APIs for communication.
• Prepared a terminal-based demonstration that can be compiled and tested in Ubuntu.

8.3 Results Achieved

The intended result is two-way message exchange between User A and User B through FIFO channels. Report successful results only after verifying them with the final code and include real terminal screenshots as evidence.

8.4 Skills and Learning Outcomes

The project provides practice with Linux terminal commands, compiling C programs, using file descriptors, understanding process isolation, creating named pipes, transferring data with read() and write(), and checking error and shutdown behaviour.

8.5 Future Enhancements

• Add a graphical user interface.
• Support more than two participants using a suitable architecture.
• Add message timestamps and optional message history.
• Improve peer-disconnection handling and error reporting.
• Add user authentication and encryption if the application is extended beyond a classroom demonstration.
• Compare FIFO communication with message queues and Unix domain sockets.

CHAPTER 9 – REFERENCES

• GNU Project. GCC online documentation: https://gcc.gnu.org/onlinedocs/
• Linux man-pages project. mkfifo(3), open(2), read(2), write(2), and close(2): https://man7.org/linux/man-pages/
• The Open Group. POSIX specifications and interfaces: https://pubs.opengroup.org/onlinepubs/9699919799/
• Ubuntu Documentation: https://help.ubuntu.com/
• Operating Systems course notes and laboratory material provided by the institution.